PET Platform

Rapport de développement

Privacy-Enhancing Technologies — Plateforme de soumission sécurisée

---

Implémentation des issues #6 et #7

Support R / Collecte d'outputs / Navigateur sécurisé Brave

Étudiant : Abdelmoumen Elimrany
GitHub : https://github.com/ElimranyAbdelmoumen/PET_Projet

Date : 22 avril 2026
Branche : `main`

Table des matières

Présentation du projet

La **PET Platform** (Privacy-Enhancing Technologies) est une application web permettant à des chercheurs de soumettre des scripts Python ou R pour exécution dans un environnement isolé. Les résultats sont vérifiés par un administrateur avant d'être accessibles à l'utilisateur.

Architecture générale

Composant	Technologie	Rôle
Backend	Flask (Python)	Interface web, authentification, API
Base de données	PostgreSQL 16	Stockage des soumissions et outputs
Worker	Python + Docker SDK	Exécution asynchrone des scripts
Sandbox	Docker (isolé)	Conteneur d'exécution sécurisé
Navigateur	Brave + Extension	Accès sécurisé à la plateforme

Le worker scrute la base de données toutes les 3 secondes, récupère les soumissions avec le statut `APPROVED`, les exécute dans un conteneur Docker éphémère, puis enregistre les résultats.

Issue #6 — Support R, sélecteur de fichiers, collecte des outputs

Objectif

L'issue demandait trois améliorations majeures de la plateforme :

1. Support de l'exécution de scripts **R** en plus de Python
2. **Sélecteur de fichier microdata** pour associer un jeu de données à une soumission
3. **Collecte et validation des fichiers de sortie** produits par les scripts

Partie 1 — Support R

Modification du runner Docker

Le fichier `runner/Dockerfile` a été mis à jour pour inclure `r-base` et `r-base-dev` (nécessaire pour compiler `rpy2`) :

Listing 1 – runner/Dockerfile

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    r-base \
    r-base-dev \
    && rm -rf /var/lib/apt/lists/*

RUN pip install --no-cache-dir \
    pandas numpy matplotlib scikit-learn scipy \
    seaborn requests openpyxl xlrd rpy2

```

```
RUN mkdir -p /work/outputs /work/data && chmod 777 /work/outputs /
    work/data
ENTRYPOINT ["python", "/usr/local/bin/run_script.py"]
```

Note : L'utilisateur `runner` (UID 10001) a été supprimé car il causait une erreur de permissions lors de l'écriture dans `/work/outputs/`.

Détection de l'extension dans le worker

Le worker (`runner_worker/worker.py`) détecte l'extension du script pour choisir l'interpréteur :

Listing 2 – Détection Python/R dans worker.py

```
ext = os.path.splitext(file_path)[1].lower() or ".py"
container_script = f"/work/script{ext}"
# ext = ".py"  => python run_script.py /work/script.py
# ext = ".R"   => Rscript --vanilla /work/script.R
```

Le script wrapper `run_script.py` route l'exécution selon l'extension :

Listing 3 – run_script.py — détection Python/R

```
if script_path.endswith(".R"):
    result = subprocess.run(
        ["Rscript", "--vanilla", script_path],
        capture_output=True, text=True, timeout=55
    )
else:
    # exec() Python avec capture des variables
    ...
```

Partie 2 — Sélecteur de fichier microdata

La table `microdata_files` stocke les jeux de données disponibles (CSV, Excel). L'utilisateur peut sélectionner un fichier microdata lors de la soumission de son script.

Schéma de base de données

Listing 4 – Tables microdata_files et submissions (extrait)

```
CREATE TABLE microdata_files (
    id        SERIAL PRIMARY KEY,
    guid      UUID DEFAULT gen_random_uuid() UNIQUE,
    name      VARCHAR(255) NOT NULL,
    file_path TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

ALTER TABLE submissions
    ADD COLUMN microdata_guid UUID REFERENCES microdata_files(guid)
    ,
    ADD COLUMN outputs_status VARCHAR(20) DEFAULT 'NONE';
```

Le fichier microdata est monté en lecture seule dans le conteneur sous `/work/data/<nom_fichier>`.

Partie 3 — Collecte et validation des outputs

Flux de validation

Étape	Description
1	Script écrit ses fichiers dans `/work/outputs/`
2	Worker scanne le répertoire local après exécution
3	Fichiers insérés dans `submission_outputs` (statut `PENDING_VALIDATION`)
4	Admin visualise et approuve ou rejette les outputs
5	Utilisateur télécharge ses fichiers si `APPROVED`

Correction du bug chemin local/hôte

Le worker utilise deux chemins distincts :

Listing 5 – Séparation chemin hôte vs chemin local

```
# Pour docker run -v (chemin vu par Docker Desktop)
host_outputs_dir = os.path.join(HOST_STORAGE_PATH, "outputs", str(
    submission_id))

# Pour os.listdir (chemin dans le container worker)
local_outputs_dir = os.path.join(STORAGE_PATH, "outputs", str(
    submission_id))

# HOST_STORAGE_PATH = /host_mnt/c/Users/User/Desktop/PET_Projet/
    storage
# STORAGE_PATH      = /storage  (mont  par docker-compose)
```

Table submission_outputs

```
CREATE TABLE submission_outputs (
    id             SERIAL PRIMARY KEY,
    submission_id INTEGER REFERENCES submissions(id),
    filename       VARCHAR(255) NOT NULL,
    file_path      TEXT NOT NULL,
    created_at     TIMESTAMPTZ DEFAULT NOW()
);
```

Issue #7 — Navigateur sécurisé Brave

Objectif

Restreindre l'accès à la plateforme au seul navigateur **Brave** avec l'extension **Brave Security Shield** installée, et activer les protections suivantes :

- Blocage du copier/coller
- Blocage des captures d'écran (PrintScreen)
- Blocage du partage d'écran (`getDisplayMedia`)

— Blocage de l'inspection (DevTools / F12)
— Filigrane (watermark) avec l'identité de l'utilisateur connecté
— Journal des violations

Architecture de l'extension

L'extension est un **Manifest V3** pour Brave/Chromium composée de :

Fichier	Rôle
`manifest.json`	Déclaration de l'extension (MV3, permissions)
`content_script.js`	Protections injectées dans chaque page
`background.js`	Service worker — stockage violations, identité
`popup.html/js`	Interface admin (toggles, watermark, journal)
`screen_block.css`	Overlay CSS de blocage visuel
`auth_bridge.js`	Pont d'authentification avec Flask

Intégration Flask

Mécanisme de détection

`content_script.js` pose un cookie `bss_active=1` à chaque visite sur la plateforme. Flask vérifie ce cookie sur toutes les routes via le décorateur `browser_required` :

Listing 6 – authz.py — décorateur browser_required

```python
def browser_required(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        if not request.cookies.get("bss_active"):
            return render_template("browser_required.html"), 403
        return fn(*args, **kwargs)
    return wrapper
```

Sans extension → HTTP 403 avec instructions d'installation.
Avec extension → accès normal.

Initialisation du watermark

Après connexion, chaque page protégée transmet l'identité Flask à l'extension via `auth_bridge.js` :

Listing 7 – Injection de l'identité dans les templates

```html
<script>
  var BSS_USER_ID = {{ session.get('user_id') }};
  var BSS_USERNAME = {{ session.get('username') | tojson }};
</script>
<script type="module" src="/static/js/bss_init.js"></script>
```

Listing 8 – bss_init.js

```python
import { initBSSAuth } from "./auth_bridge.js";
if (BSS_USER_ID) {
```

```
  initBSSAuth({ uid: String(BSS_USER_ID), displayName: BSS_USERNAME
    });
}
```

Le watermark affiche alors le nom de l'utilisateur connecté en filigrane diagonal sur toutes les pages.

Installation de l'extension

1. Installer **Brave Browser** (https://brave.com)
2. Ouvrir `brave://extensions` et activer le **mode développeur**
3. Cliquer sur *Charger l'extension non empaquetée*
4. Sélectionner le dossier `breavescripts/` du projet
5. Copier l'**Extension ID** et le remplacer dans `auth_bridge.js` ligne 37

Benchmark du sandbox

Conditions d'exécution

— **Image :** `portwatch-python-runner:1.0`
— **Limites :** 512 Mo RAM, 1 CPU, réseau désactivé (`-network none`)
— **Timeout :** 60 secondes
— **Méthode :** 5 exécutions par scénario, mesure du temps total docker run
— **Machine :** Windows 11, Docker Desktop (WSL2 backend)

Résultats

Table 1 – Temps d'exécution du sandbox (ms) — 5 runs par scénario

Scénario	R1	R2	R3	R4	R5	Moy.
Python — hello world	1611	2070	1417	1689	1379	**1633**
Python — pandas describe	1347	1502	1351	1271	1361	**1366**
Python — sklearn LinearReg.	1395	1514	1253	1484	1427	**1414**
Python — lecture microdata CSV	1293	1593	1395	1545	1737	**1512**
Python — écriture output CSV	1570	1601	1501	1344	1398	**1482**
R — hello world	1613	1613	1375	1423	1344	**1473**
R — stats rnorm(1000)	1743	1483	1339	1600	1484	**1529**
R — lecture microdata CSV	1424	1759	1631	1464	1332	**1522**

Analyse

Temps de démarrage du conteneur. Le temps dominant (~1,3–1,7 s) correspond au démarrage du conteneur Docker éphémère, pas au calcul. Le code lui-même s'exécute en quelques millisecondes.

Python vs R. Les deux langages ont des performances quasi identiques dans ce sandbox (différence < 200 ms). La variabilité observée est liée à la charge système, pas à l'interpréteur.

Overhead microdata et outputs. L'ajout d'un fichier microdata ou d'une écriture de fichier output ajoute moins de 150 ms — overhead négligeable.

Stabilité. Aucun timeout ni erreur sur 40 exécutions. L'écart-type estimé est de ±150 ms, cohérent avec la variabilité du démarrage Docker.

Synthèse des contraintes

Contrainte	Valeur	Respectée
Isolation réseau	`-network none`	
Mémoire max	512 Mo	
CPU max	1 core	
Timeout	60 s (moy. ~1,5 s)	
Support Python	pandas, numpy, sklearn...	
Support R	stats, utils, base	
Lecture microdata	`/work/data/`	
Écriture outputs	`/work/outputs/`	

Conclusion

Les deux issues demandées par l'encadrant ont été implémentées et mergées sur la branche `main` :

- **Issue #6** — Support complet R + sélecteur microdata + collecte/validation des outputs avec workflow admin
- **Issue #7** — Intégration de l'extension Brave Security Shield : accès interdit sans l'extension (HTTP 403), watermark utilisateur, blocage copier/coller, screenshots et DevTools

Le sandbox Docker isole correctement chaque soumission. Le coût fixe de démarrage (~1,5 s) est largement acceptable pour une plateforme de traitement de code scientifique.

Abdelmoumen Elimrany
22 avril 2026